This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Don't Let Your System Decide It's Dead

Aaron H. Alpar
Independent Researcher

Abstract

Distributed systems fail in two ways: the failure itself, and the system's automatic response to the failure. The second is often worse. This paper derives a computable boundary h^* that separates provably safe automatic recovery from provably intractable recovery under production load. The boundary emerges from a three-level *recovery invariant*—individual cost boundedness, aggregate rate boundedness, and feedback stability—formalized through the recovery gain matrix $\mathbf{G}$ and its spectral radius $\rho(\mathbf{G})$. When headroom exceeds h^*, recovery is a proven contraction; when it does not, the system's correct action is to emit diagnostic state and shut down. We compute closed-form h^* for UNIX swap thrashing (cross-resource coupling, smooth stability transition) and Cassandra hint replay (M/M/1 phase transition: gain jumps from 0 to 3,000 at 1% remaining disk headroom). TCP's exponential backoff serves as a degenerate reference ($G = -0.5$, unconditionally stable). The OOM killer illustrates the opposite extreme: it cannot compute h^* because it lacks a resource model for restart cost. The *compensation boundary*—the architectural point where a system stops automating recovery and starts emitting state—should be drawn where the provability boundary actually is. We apply the framework prescriptively to CRDT tombstone garbage collection.

1 Introduction

Distributed systems fail in two ways: the failure itself, and the system's automatic response to the failure. The second is often worse.

The root cause is systems making semantic decisions with insufficient information—promoting "slow" to "dead" based on wall-clock constants rather than measured resource models. There is a precise invariant that separates safe automatic recovery from cascading failure, and a design principle that tells you who should make which decisions. Most production systems violate both.

2 The Setup: A Configuration Parameter That Eats Your Data

Cassandra has a setting called `gc_grace_seconds`. It defaults to 864,000—ten days. Here is what it does: when you delete a row, Cassandra does not remove it. It writes a *tombstone*—a marker that says "this key was deleted." Tombstones exist because in an eventually consistent system, if you just remove the data, another replica that has not heard about the deletion will happily replicate the old value back. The tombstone is the deletion's proof of existence.

`gc_grace_seconds` is how long Cassandra keeps that proof around. After ten days, Cassandra garbage-collects the tombstone. The assumption: repair has propagated the tombstone to all replicas within that window.

If repair did not run—or did not finish—within those ten days, the tombstone disappears from some replicas while the pre-delete value still exists on others. The deleted data silently reappears. This is a well-documented production failure mode.

What just happened? A single wall-clock constant made two different decisions simultaneously:

1. **A local decision**: "This tombstone's storage can be reclaimed." (Garbage collection—the system has the information to do this.)

2. **A semantic decision**: "All replicas have observed this deletion." (Replication state—the system is *guessing*.)

These are different decisions requiring different information. Conflating them is the root of most cascading failures in distributed systems, and the pattern repeats everywhere: timeouts that promote "slow" to "dead," OOM killers that choose which process to sacrifice, swap systems that silently trade memory bandwidth for disk bandwidth until everything grinds to a halt.

This paper is about drawing the line correctly.

3 The Two Decisions

Every failure in a distributed system requires two decisions:

1. **The local decision**: "This connection/request/operation has failed."

2. **The semantic decision**: "This node/peer/service is gone and I should act accordingly."

The first is a statement about what the system has observed. The second is an interpretation of what that observation *means*. They require different information, are made at different layers, and have different consequences when wrong.

Getting the local decision wrong is usually cheap—a spurious timeout, a wasted retry. Getting the semantic decision wrong can be catastrophic—deleted data resurrected, a healthy node declared dead and its traffic redistributed into an already-overloaded cluster, a cascade that turns a minor hiccup into a major outage.

The engineering challenge is that the semantic decision is *tempting* to automate. Manual intervention is slow. Operators are expensive. And most of the time, the guess is right—the node really is dead, repair really did finish, the process really was misbehaving. Systems are designed for the common case, and the common case makes automation look safe. The failure modes live in the gap between "usually right" and "always right."

4 TCP Got This Right (Mostly)

TCP's retransmission timeout is the gold standard for automatic failure decisions. It gets nearly everything right, and examining *why* reveals the principles that other systems violate.

Derived from measurement, not magic constants. TCP's retransmission timeout (RTO) is computed from Jacobson's round-trip time (RTT) estimator—a smoothed average plus variance of measured round-trip times. The timeout tracks actual network behavior, not an engineer's guess about what it should be.

Individual recovery cost is bounded and known. A single retransmission costs one segment—fixed, known in advance, expressed in the same units as normal operation.

Recovery never amplifies the failure. Exponential backoff ensures the retry *rate* converges to zero. Each successive timeout doubles the wait. The more congested the network, the less recovery traffic TCP adds to it.

Failure is surfaced, not hidden. When TCP exhausts its retries, the application gets `ETIMEDOUT`. TCP does not guess what the timeout *means*—it tells the application "this connection is dead" and lets the application decide whether to reconnect, try a different server, or alert an operator.

TCP separates the layers cleanly:

- **TCP decides**: "This connection is dead." (It has the information: measured RTT, retry history, bounded retransmission cost.)
- **Application decides**: "What does this dead connection mean?" (It has the context: service topology, redundancy, retry policy.)
- **Neither decides**: "This node is permanently gone." (This requires physical-world knowledge—hardware state, operator intent—that no software layer has.)

There is a nuance most people miss: TCP *does* eventually give up automatically. `tcp_retries2` defaults to 15 retries, which works out to roughly 13–30 minutes depending on measured RTT. This is a wall-clock timeout, but grounded in a measured resource model—derived from observed network behavior, not pulled from thin air.

4.1 Why This Separation Is Provably Necessary

4.1.1 The Impossibility Result

The FLP impossibility result [6] proves that in an asynchronous system where at least one process may crash, no deterministic algorithm can guarantee consensus. The load-bearing assumption is *asynchrony*: with no bound on message delay or processing time, you cannot distinguish a crashed process from a slow one.

TCP operates *within* the bounds where this distinction is tractable—it measures RTT, so it has a probabilistic model of "how slow is normal." But the question "is this node permanently gone?" lives in the asynchronous regime where FLP applies. No amount of measurement resolves it, because the relevant information (hardware failure, network partition, operator intent) is outside the system's observation boundary.

This is the theoretical foundation for the rest of the argument: systems that promote "slow" to "dead" automatically are not just making a bad engineering choice—they are making a decision that is *provably underdetermined* by the information available to them. The correct response is the one TCP models: decide what you can (connection liveness), surface what you cannot (`ETIMEDOUT`), and let the layer with more information make the semantic call.

4.1.2 The Workaround

Chandra and Toueg [3] formalized the workaround: you can solve consensus if you have a *failure detector* with known properties (completeness and accuracy). Different detector strengths enable different guarantees. This maps directly to the layered model above—each layer is a failure detector with a specific information boundary, and the system's correctness depends on no layer exceeding its boundary.

5 The Recovery Invariant

TCP's design is not just good engineering intuition—it satisfies a precise invariant that separates safe automatic recovery from cascading failure. The invariant has three levels. Each is necessary. When all three hold, recovery converges locally—within the linearization radius of the current operating point (section A). The monotonicity result (section A) ensures the computed h^* is a lower bound on the true stability boundary: the framework never declares safety when danger exists, though it may declare danger when safety exists. Section F bounds the gap to the nonlinear system.

5.1 Precondition: You Need a Resource Model

Before evaluating any level, the system should answer a basic question: *recovery costs what, measured in what units, against what capacity?* Systems that lack a complete resource model can still be evaluated informally against levels 1–3, but the results are weaker—statements about conceptual boundedness rather than measured guarantees.

The system must name its bottleneck resource, measure its total capacity, and express both normal operations and recovery actions in the same units against that capacity.

TCP has an explicit resource model: bandwidth, measured in segments per round-trip. One normal packet and one retransmission cost the same unit—one segment—against a known link capacity.

Cassandra's hinted handoff has a partial model: one hint replay costs roughly one write in I/O operations per second (IOPS). But the available IOPS during replay depend on concurrent read load, compaction, and repair—variables the hint replay system does not observe.

The OOM killer has almost no resource model: it knows system-wide memory pressure triggered the kill, but the cost of killing a specific process (how much memory is freed, how long until the supervisor restarts it, what the restart will allocate) depends on external state the kernel does not track.

The precondition is what separates systems that *can* reason about recovery cost from those that act and hope.

5.1.1 Formal Framework: Bandwidth as Primitive

Every system resource has a consumption rate expressible as bandwidth: **units of resource per unit time**.

Resource	Bandwidth unit
Network	bytes/sec or segments/RTT
Disk I/O	IOPS or bytes/sec
CPU	utilization (dimensionless rate)
Memory	pages/sec (fault-in rate)

For pure-flow resources (network, CPU), bandwidth *is* the resource. There is no at-rest state—a CPU cycle is consumed or not, a packet is in flight or not.

For stock resources (memory capacity, disk space), the at-rest state is the integral of past bandwidth:

$$S(t) = \int_0^t b_{\text{net}}(\tau)\, d\tau \tag{1}$$

where b_{net} is the net consumption rate (allocation rate minus free rate). Every physical memory page was populated through a page fault—zero-fill or disk read—that consumed memory bus bandwidth

and potentially disk bandwidth. Virtual address space can be reserved without bandwidth (`malloc` with overcommit returns immediately), but physical resource consumption always goes through bandwidth.

This unification matters: the recovery invariant constrains rates. Stocks enter only as boundary conditions on accumulated rates.

5.1.2 Multi-Resource Model

A system has n resources. At time t:

- C_i: capacity of resource i (bandwidth ceiling, in units$_i$/sec)
- $w_i(t)$: bandwidth consumed by normal operations on resource i
- $r_i(t)$: bandwidth consumed by recovery actions on resource i
- $h_i(t) = C_i - w_i(t) - r_i(t)$: headroom on resource i

These form vectors $\mathbf{C}$, $\mathbf{w}(t)$, $\mathbf{r}(t)$, $\mathbf{h}(t) \in \mathbb{R}^n$. For stock resources, an additional integral constraint applies:

$$\int_0^t b_{i,\text{net}}(\tau)\, d\tau \leq S_i \tag{2}$$

Recovery is safe when both conditions hold on every resource:

1. $h_i(t) \geq 0$ at all t (bandwidth not exceeded)
2. $\int_0^t b_{i,\text{net}}(\tau)\, d\tau \leq S_i$ for stock resources (capacity not exhausted)

The bottleneck is whichever resource hits either constraint first. It is emergent, not fixed—recovery actions can shift the bottleneck from one resource to another.

5.2 Level 1: Individual Bound

> Each recovery action's resource cost must be bounded and known.

This is the weakest condition. You must know what a single compensating action costs before you trigger it.

TCP satisfies this: one retransmission costs one segment—fixed, known, independent of system state. Cassandra's hinted handoff satisfies this too: one hint replay costs approximately one write. The OOM killer satisfies this: one kill frees the memory of one process.

Every system discussed in this paper passes level 1. That is why it is insufficient. A system can have individually cheap recovery actions and still cascade—the danger is in their aggregate behavior.

5.3 Level 2: Aggregate Bound

> At any point in time, the aggregate resource consumption by all concurrent recovery actions must remain within the headroom not consumed by normal operations: $h_i(t) = C_i - w_i(t) - r_i(t) \geq 0$ for every resource i.

This is the burst dimension. The distinction between "amortized OK" and "worst-case catastrophic" lives here.

TCP satisfies this: exponential backoff ensures the *rate* of retransmissions decreases with each round, so aggregate retry bandwidth shrinks over time. Google's SRE retry budget is an explicit

level-2 mechanism—capping retries at 10% of requests bounds the aggregate cost to 1.1× normal load regardless of failure count.

Cassandra's hinted handoff violates this: hints accumulate linearly during a node's absence and replay in a burst on return. A 10-minute outage at 100 Mbps per node produces ∼7 GiB of hints; at the default 1024 KB/s throttle, replay takes ∼2 hours. If the throttle is raised (as operators under pressure often do), the burst can recreate the overload that caused the original failure.

5.4 Level 3: Feedback Bound

> Recovery actions must not create conditions that trigger further failures. Equivalently: the recovery feedback loop must have gain < 1.

Levels 1 and 2 treat recovery actions as independent events. Level 3 asks whether they interact. A system violates level 3 when recovery actions consume the same resources that are under contention, creating a positive feedback loop: failure → recovery → more contention → more failure.

TCP satisfies this: backoff *reduces* congestion on the link, which reduces the condition (congestion) that caused the timeout in the first place. The feedback loop is damped—each iteration moves the system closer to equilibrium, not further.

The OOM killer can violate this: a process supervisor restarts the killed process, which allocates memory, which recreates the pressure that triggered the kill. The feedback loop gain depends on external configuration (supervisor policy) that the kernel does not know about—another instance of a system making decisions outside its information boundary (section E formalizes this gain ratio and shows why the kernel cannot evaluate it).

Cassandra's hinted handoff can violate this: burst replay causes CPU/IO contention → read latencies spike → read timeouts trigger read repairs → read repairs consume more CPU/IO → more timeouts. The recovery action for writes triggers a cascade in the read path.

The decompensation cascade is what a level 3 violation looks like in practice:

1. System overloaded → operations slow down
2. Timeouts fire → compensating actions triggered
3. Compensating actions consume the bottleneck resource → more contention
4. More timeouts → more compensating actions → feedback gain ≥ 1 → cascade

Systems designed for "graceful degradation" often degrade ungracefully because the compensating actions were designed for the steady-state cost model (level 1 holds, level 2 holds at low utilization), not the failure cost model (level 2 breaks under burst, level 3 breaks under sustained pressure). David Woods [19, 20] calls this *decompensation*—the exhaustion of adaptive capacity when the system's own recovery mechanisms become the dominant source of load.

The metastable failures literature [2, 13] identifies the same phenomenon: "the root cause of a metastable failure is the sustaining feedback loop, rather than the trigger." Their taxonomy of triggers (load-spiking vs. capacity-decreasing) and amplification mechanisms (workload amplification vs. capacity degradation) maps directly onto levels 2 and 3 of this invariant. The formalization in section A approaches the same problem from a different mathematical tradition—control theory and linear algebra rather than Markov chains and queuing theory.

A note on quiesced recovery. Some systems sidestep levels 2 and 3 entirely by deferring recovery to a state where no production traffic competes for resources—replaying a write-ahead log on startup before accepting connections, running repair during a maintenance window, or batching reconciliation while the node is offline. When the competing workload is zero, level 2 is trivially satisfied and level 3's feedback loop is broken. This is effectively admission control: the system refuses new work until recovery is complete. The trade-off is availability—the node is down during recovery.

5.5 Summary

Level	Condition	TCP	Hinted handoff	OOM killer
0	Resource model: C_i, $w_i(t)$, $r_i(t)$ in shared bandwidth units	✓ segments on a link	× partial: IOPS but not concurrent load	× memory freed unknown until kill
1	Individual: cost of one action bounded and known	✓ one segment	✓ one write	✓ one kill
2	Aggregate: recovery rate $\leq$ freed capacity	✓ exponential backoff	× burst on return	per-event only
3	Feedback: recovery gain < 1	✓ backoff reduces congestion	× replay triggers read repair cascade	× supervisor restarts killed process

The recovery invariant tells you whether recovery *is* safe. It does not tell you whether it will *stay* safe, or what to do when it is not. A system can satisfy all three levels at current load and violate them minutes later. That question—when to compensate automatically and when to stop and escalate—requires a different concept.

6 The Compensation Boundary

> The *compensation boundary*[1] is the point in a system's architecture where it stops surfacing failure to a higher authority and starts acting on it automatically. Below the boundary, errors are reported—the caller decides. Above it, the system compensates silently—no one is asked.

The placement of that boundary determines how the system fails under pressure.

6.1 z/OS: The Boundary Drawn Furthest Out

z/OS draws the compensation boundary furthest out of any system I know. WLM assigns work to service classes with defined goals; when the system cannot meet goals, it reports the shortfall rather than silently degrading.[2] z/OS manages memory through explicit region sizes—exceed yours and the job abends (S878, S80A), surfacing a hard failure to the operator.

[1] The term *compensating action* originates in transaction processing, where a compensating transaction undoes the effects of a committed transaction that cannot be rolled back [7]. *Decompensation*—the exhaustion of adaptive capacity when recovery mechanisms become the dominant load source—is David Woods' contribution (see section 10). The *compensation boundary* as used here—the architectural point where the system stops surfacing failure and starts acting on it—is a framing specific to this paper, combining the transactional concept (what is a compensating action?) with the resilience engineering concept (when does compensation become the problem?) and adding the design question: *where should the system draw that line?*

[2] WLM (Workload Manager) is the z/OS component that classifies work into service classes with defined performance goals (response time, velocity, discretionary) and dynamically manages system resources—CPU dispatching

XCF heartbeat failure detection declares a member dead (it has the information: heartbeat history, configurable intervals), but does not decide what the failure *means* for application-level data—recovery exits handle that decision, and may themselves escalate.[3] Automation is policy-driven (SA z/OS): an operator authored the rule, every automated action is logged, and when no policy matches, the system escalates rather than guessing.[4]

The escalation mechanism itself is significant: WTO/WTOR (Write to Operator with Reply) is *synchronous*—on critical decisions, the system issues a message and **waits for the operator to respond** before proceeding. The operator is in the control loop by design—consulted during the decision, not notified afterward.

But z/OS is not immune. Under system-wide memory pressure, it pages to auxiliary storage silently. SRM—which coexists with WLM as its resource-allocation mechanism—swaps out entire address spaces based on system-wide monitoring (paging rates, UIC values, central storage demand). WLM sets policy goals; SRM decides the mechanics. That separation is itself an example of decision authority tracking information: WLM knows what matters to the workload, SRM knows what is happening to the hardware. But the swap-out is still an automatic decision about who gets to run. The compensation boundary is further out than UNIX's, not absent.

6.2 UNIX: A System at War with Itself

UNIX draws the boundary closer in, and the result is a system with a split personality.

The syscall interface surfaces errors faithfully—`errno`, `ENOSPC`, `ENOMEM` from `fork`, `SIGPIPE`, exit codes. This is the "let the caller decide" philosophy. But the resource management layer behind it hides scarcity, and virtual memory is the clearest example.

When physical memory is exhausted, UNIX pages to disk—a compensating action designed to degrade rather than fail. The capacity model is straightforward: physical RAM plus swap, both measurable in bytes, upper bound known at boot. But the *cost* model is incoherent. The system is not trading memory for disk *space* (swap is pre-allocated); it is trading memory bandwidth for disk bandwidth. A memory access costs ∼100 ns; a page fault to an HDD costs ∼10 ms; to NVMe, ∼100 µs. The compensating action substitutes a resource that is 1,000–100,000× slower per access, and the only unit that makes both sides commensurable is time—the one unit the system does not budget.

Against the recovery invariant: Level 1 holds—one page swap frees one page of physical memory, bounded and known. Level 2 is conditional—it holds while disk I/O remains below saturation, but the system swaps without checking disk bandwidth. Level 3 breaks under pressure: swapping consumes both memory bandwidth (page copies) and disk bandwidth (I/O operations), slowing everything → queues build → more memory allocated to pending work → more swapping. Thrashing is a level 3 violation—the recovery action amplifies the condition it compensated for.

This is a defensible decision within UNIX's information boundary—the kernel knows about

priority, I/O priority, memory—to meet those goals. When goals cannot be met, WLM reports the shortfall rather than silently degrading. See IBM, *z/OS MVS Planning: Workload Management*, SC34-2662.

[3] XCF (Cross-system Coupling Facility) provides group membership, signaling, and heartbeat-based failure detection across z/OS systems within a sysplex. XCF monitors member health via configurable heartbeat intervals and declares a member failed when heartbeats stop—a local decision about connectivity. What that failure *means* for application-level state is delegated to recovery exits registered by the application, not decided by XCF itself. See IBM, *z/OS MVS Setting Up a Sysplex*, SA23-1399.

[4] SA z/OS (IBM System Automation for z/OS) is a policy-driven automation framework for z/OS operations. Operators author automation rules that define responses to system events; every automated action is logged and auditable. When no rule matches a condition, SA z/OS escalates to the operator rather than guessing. See IBM, *System Automation for z/OS Planning and Installation*, SC34-2571.

memory and disk, and paging is within its operational scope. It makes no semantic guess about the application. But acting within your boundary does not guarantee stability. Swap satisfies the invariant at low memory pressure and violates it under high pressure, and the system has no mechanism to predict the transition. The implicit stabilization mechanism is degradation itself: as everything slows, users give up, load drops. This is the *absence* of admission control, not a form of it.

The OOM killer is where the contradiction becomes explicit. When oversubscription hits the wall, Linux makes an automatic semantic decision with unbounded consequences—it kills a *different* process than the one that caused the pressure, chosen by heuristic (`oom_badness` scoring, primarily by memory footprint). Even the default overcommit mode (mode 0, heuristic) allows this; mode 1 (`vm.overcommit_memory=1`) is starker: `malloc` *always* succeeds regardless of available memory, then the kernel kills processes later when it cannot honor the promise. Kubernetes sets mode 1 by default.

The error is not surfaced—it is deferred until it is unrecoverable. When the OOM killer fires, it logs the kill to the kernel ring buffer and continues—no operator acknowledgment, no pause before acting.

UNIX does have tools that push the boundary outward: disk quotas (`EDQUOT`), ulimits (`SIGXCPU`, `SIGXFSZ`, `EMFILE`), cgroups (scoped OOM, CPU throttling). When configured, these behave like the mainframe model—the system says "no" and surfaces an error. But they are all opt-in. A fresh UNIX system has no disk quotas, generous ulimits, no cgroups. Compare z/OS: you cannot submit a job without a region size; WLM service classes are mandatory. The compensation boundary in UNIX is a configuration choice. In z/OS it is a design invariant—the system will not run without it. Under production pressure, opt-in boundaries tend to be the first thing relaxed or forgotten.

6.3 Dynamo-Family Systems: No Boundary at All

Dynamo-family systems (Cassandra, Riak, early DynamoDB) barely have a compensation boundary. Compensating actions are implicit in each subsystem, not centrally authored or auditable. Cassandra's `gc_grace_seconds` promotes "old" to "replicated everywhere" based on a wall-clock constant. Hinted handoff accumulates hints during a node failure and replays them in a burst on return. No escalation path exists—when the automatic decision is wrong, the system has already acted.

6.4 The Pattern

	Reports failure	Compensates silently	Escalation model	Structural?
z/OS	Abends, WLM goal misses, console messages	Paging under system-wide pressure	Synchronous—WTO/WTOR waits for operator	Yes—cannot run without region sizes, WLM
UNIX	`errno` at syscall boundary	VM overcommit, swap, OOM killer	Asynchronous—syslog/dmesg, after the fact	No—quotas/cgroups exist but are opt-in
Dynamo	Logs (if watching)	Everywhere: GC, hinted handoff, read repair	None—no escalation path	No—compensation is the design

As the compensation boundary moves inward, the system's failure modes shift from *recoverable* (operator gets a message, decides what to do) to *unrecoverable* (data silently lost, wrong process killed, cascade already in progress). The key property is not whether a system automates—z/OS automates extensively—but whether *wrong* automatic decisions are surfaced before their consequences

become permanent.

The historical sequence matters. IBM's OS/360 (1964) established mandatory compensation boundaries—region sizes, operator-in-the-loop escalation via WTO/WTOR—before UNIX existed. UNIX's designers chose a different philosophy: permissive defaults, graceful degradation, opt-in resource limits. The distributed systems community inherited UNIX's choice wholesale. The formal framework presented here provides the mathematical basis for IBM's original design: there is a computable boundary h^* below which automated recovery is provably safe and above which it is provably intractable under production load. IBM enforced this boundary by convention; the framework shows it can be derived from measured system parameters.

7 Decision Authority Tracks Recovery Fitness

The design principle that ties the preceding sections together:

> A system may automate recovery when it can demonstrate that recovery bandwidth fits within available headroom—that $h(t) > h^*$ for its current state. This is a computable condition: the gain matrix $\mathbf{G}$ and the spectral radius $\rho(\mathbf{G})$ provide the test. When the condition holds, automated recovery is a proven contraction. When it does not hold, recovery under production load is intractable. The system's correct action is to emit diagnostic state and shut down, deferring recovery to an offline context where production load is zero and the bandwidth constraint is trivially satisfied.

Decision	Can prove recovery fits?	Reversal cost if wrong	Action
TCP retransmission	Yes ($G = -0.5$, contraction)	N/A (always converges)	Automate
Remove backend from LB	Yes (re-add: L1–L3 trivial)	Bounded: seconds of wasted capacity	Automate
Cassandra hint replay	Depends: safe when $R < R^*$	Offline reversible: must quiesce	Automate if L2-throttled; shut down otherwise
OOM kill	No (restart cost unknown, L0 violation)	Unbounded	Emit state, shut down
Tombstone GC	No (membership unknown, FLP)	Irreversible: zombie resurrection	Escalate to operator

The governing variable is whether the system can prove—from its gain matrix and current headroom—that recovery converges. A load balancer removing a backend can prove this: re-adding costs one config update, satisfying L1–L3 trivially. Tombstone GC cannot: the consequences of being wrong (zombie resurrection) have no recovery action at all. Between these extremes, Cassandra hint replay illustrates the critical middle case: the recovery action is correct, but its bandwidth cost may exceed available headroom. The system must either throttle to fit (L2 bound) or shut down and

recover offline.[5]

8 Applying the Principles: CRDT Tombstone GC

CRDT tombstone garbage collection is a clean test case for these principles, because it forces every question they raise.[6]

CRDTs track deletions with tombstones—markers that say "this key was deleted at time T." A tombstone can only be GCed when *every* replica has observed the deletion, because removing it prematurely from any replica lets the pre-delete value reappear (the zombie resurrection problem). "Every replica has observed it" requires knowing two things: the group membership, and each member's progress. The first is a semantic fact about the physical world. The second is measurable.

The slow-vs-dead problem is inescapable here. A peer stops acknowledging—is it partitioned, overloaded, or decommissioned? This is exactly the FLP regime: no amount of observation from inside the system resolves it. Any system that GCs tombstones automatically must promote "unresponsive" to "gone"—an automatic semantic decision with unbounded consequences (silent data resurrection if wrong).

Cassandra's approach crosses every boundary. `gc_grace_seconds` is a wall-clock constant that assumes repair will propagate tombstones within 10 days. This is a semantic decision disguised as a configuration parameter, made automatically by the system with insufficient information. The recovery action—hinted handoff replay on node return—violates the recovery invariant by bursting accumulated writes into an already-stressed cluster.

What the principles demand instead:

1. **Separate what the system can measure from what it cannot.** The system *can* measure: how many tombstones exist, how old they are, which peers have acknowledged which state, and how far behind each peer is. The system *cannot* determine: whether an unresponsive peer will return.

2. **Surface the measurable; defer the unmeasurable.** The API exposes what the system knows: `TombstoneCount()` reports pressure—how urgently GC is needed. `Status()` reports per-peer replication lag—who is behind and by how much. `BlockedBy(d)` names the specific peers preventing a given deletion from being GCed—the bottleneck, not just the symptom.

3. **Reserve the semantic decision for the entity with sufficient information.** `RemovePeer()` is the only call that declares a node gone. It requires the operator—the one entity that can check whether the hardware is dead, whether the network is partitioned, whether the node is being decommissioned. The system will not make this call on its own.

4. **Make the recovery action satisfy the invariant.** If a removed peer returns, it reconciles via *asymmetric* Merkle-tree anti-entropy [4]—comparing hash trees with each replica,

[5] DHCP lease expiry faces the same structural problem—"lease expired" is not "address unused"—but largely avoids it through protocol design: the client contractually agrees to stop using the address at expiry, and Duplicate Address Detection (RFC 5227) catches most residual conflicts. The type error exists but the protocol mitigates it bilaterally rather than ignoring it. Contrast Cassandra's `gc_grace_seconds`, where no such mitigation exists—the wall-clock constant is the entire mechanism.

[6] CRDT (Conflict-free Replicated Data Type)—a family of data structures that can be replicated across multiple nodes, updated independently and concurrently, and merged deterministically without coordination. The merge function is mathematically guaranteed to converge (it forms a join-semilattice), which eliminates the need for consensus on every write—at the cost of carrying metadata, including tombstones, that can only be safely discarded when all replicas have observed it. See Shapiro et al. [17].

identifying divergent ranges, and pulling only the cluster's current state for those ranges. Reconciliation is pull-only: the returning node does not push stale data back to the cluster, preventing zombie resurrection for keys whose tombstones were GC'd after removal. No stored delta log is needed—divergence is discovered by comparing current state directly. Cost is $O(|diff|)$: proportional to actual divergence, not to state size or absence duration. The returning node pulls at a rate bounded by level 2 headroom—available capacity minus current load—so the cluster does not experience a burst. The framework's own invariant defines the throttle: reconciliation proceeds while $r(t) < C(t) - w(t)$ and pauses when headroom is exhausted.

The result is a compensation boundary drawn where the provability boundary actually is: the system automates recovery when it can prove recovery bandwidth fits within headroom (L2-throttled Merkle reconciliation), and surfaces everything else to the operator (peer membership decisions). No wall-clock constant substitutes for operator judgment. No automatic action has consequences that exceed available headroom.

9 Design Rules

1. **Automate decisions whose reversal satisfies the recovery invariant. Escalate decisions whose consequences are irreversible or unbounded.** TCP can kill a connection (reversal: reconnect, L1–L3 trivial). Only an operator can decommission a node (reversal: rebuild, offline only).

2. **Derive timeouts from measured properties.** If you cannot measure it, you cannot timeout on it safely.

3. **Check all three levels of the recovery invariant.** Individual cost bounded (level 1) is necessary but insufficient. Verify that aggregate recovery rate stays within freed capacity (level 2) and that recovery actions avoid amplifying the failure condition (level 3). Level 3 failure is a cascade seed.

4. **Surface state, do not hide it.** When you cannot decide safely, expose what you know to whoever can.

5. **Make failure a first-class API.** TCP does not hide retransmissions from the application—it surfaces `ETIMEDOUT`. Your system should not hide peer lag from the operator.

6. **If you cannot prove L1–L3 under current load, emit diagnostic state and shut down.** Do not attempt recovery that may violate the invariant. Do not continue operating in a state you cannot characterize. Shutdown is recoverable. Silent cascade is not.

10 Related Work

The phenomenon of recovery actions amplifying failures is well-identified across multiple communities.

Metastable failures. Bronson et al. [2] coined the term and identified the sustaining feedback loop—not the trigger—as the root cause. Huang et al. [13] studied 22 metastable failures across 11 organizations, finding that >50% involved retry storms. Isaacs and Alvaro [14] propose a modeling pipeline for discovering susceptibility at design time. The formal analysis by Alvaro et al. [1] provides a CTMC-based spectral characterization of metastable states. This paper approaches the same phenomenon from control theory rather than stochastic processes: the gain matrix asks "does the perturbation converge or diverge?" where CTMCs ask "how long does the system stay trapped?"

Resilience engineering. David Woods' concept of *decompensation*—the exhaustion of adaptive capacity when recovery mechanisms become the dominant source of load [19, 20]—captures the qualitative phenomenon that level 3 of the recovery invariant formalizes quantitatively. Woods asks whether the system can extend capacity at all (*graceful extensibility*); the compensation boundary asks *where* the system draws the line between reporting and acting.

Control theory for computing. Hellerstein et al. [11] applied control theory (MIMO, transfer functions, gain, pole placement) to computing systems. Their focus is on designing controllers (admission control, scheduling), not on analyzing when existing recovery mechanisms cascade. The gain matrix formalization in this paper uses the same mathematical tradition for a different question.

Cascading failures in other domains. Ramirez, Odijk, and Bauso [16] applied gain matrix and Lyapunov stability analysis to cascading failures in financial networks. The spectral radius stability condition $\rho(G) < 1$ is the same; the domain and the specific construction of G differ.

Failure detectors. Chandra and Toueg [3] formalized the properties (completeness, accuracy) that failure detectors need to enable consensus, establishing the theoretical foundation for the layered decision authority model.

Exponential backoff stability. The formal stability of binary exponential backoff is well-studied. This paper uses TCP as a pedagogical reference design—an existence proof that safe automatic recovery is achievable—rather than making new claims about TCP itself.

TCP congestion control stability. Low, Paganini, and Doyle [15] analyzed the stability of TCP congestion control (Reno, Vegas, FAST) using linearization and spectral conditions on the coupled system of N flows sharing M links—the same mathematical framework used here. Their stability conditions depend on link delays, flow counts, and AQM parameters. Our scalar $G = -0.5$ characterizes the backoff *mechanism*'s contraction ratio—each congestion event halves recovery demand for an individual flow—not the stability of the composite TCP system. Our contribution is a framework for comparing mechanism-level gain properties across heterogeneous recovery systems (TCP, swap, Cassandra, OOM), not a new result on TCP congestion dynamics.

A The Gain Matrix: Formalizing the Feedback Bound

We model recovery dynamics near the operating point as a first-order linearized system. The state is the headroom vector $\mathbf{h}(t)$, and the feedback structure is captured by the recovery gain matrix.

Define the $n \times n$ recovery gain matrix $\mathbf{G}$:

$$G_{ij} = \frac{\partial r_i}{\partial(-h_j)} \tag{3}$$

Entry G_{ij} measures how much additional recovery bandwidth appears on resource i when headroom on resource j decreases by one unit. For systems without backoff, $G_{ij} \geq 0$: less headroom means equal or greater recovery demand.

When headroom drops by $\Delta\mathbf{h}$, recovery demand changes by $\mathbf{G} \cdot \Delta\mathbf{h}$. For $\mathbf{G} \geq 0$ this is an increase (amplifying); for entries $G_{ij} < 0$ (e.g., backoff) it is a decrease (stabilizing). In either case the iteration proceeds:

$$\Delta\mathbf{h} \to \mathbf{G} \cdot \Delta\mathbf{h} \to \mathbf{G}^2 \cdot \Delta\mathbf{h} \to \cdots \tag{4}$$

$$\text{Total: } (\mathbf{I} + \mathbf{G} + \mathbf{G}^2 + \cdots) \cdot \Delta\mathbf{h} = (\mathbf{I} - \mathbf{G})^{-1} \cdot \Delta\mathbf{h} \tag{5}$$

This geometric series converges if and only if $\rho(\mathbf{G}) < 1$, where $\rho(\mathbf{G})$ is the spectral radius of $\mathbf{G}$ (the largest absolute value among its eigenvalues).

Stability condition: $\rho(\mathbf{G}) < 1$.

For a single resource, $\mathbf{G}$ is scalar and the condition reduces to $|G| < 1$. For multiple resources, the spectral radius captures cross-resource feedback: even if each diagonal entry $G_{ii} < 1$, the system cascades when off-diagonal entries are large enough—$G_{12} \cdot G_{21} \geq 1$ means resource 1's recovery stresses resource 2, whose recovery stresses resource 1.

A.1 Monotonicity of the Stability Boundary

For systems with $\mathbf{G} \geq 0$, the spectral radius is non-decreasing in every entry of $\mathbf{G}$.[7] If load increase causes any G_{ij} to grow, $\rho(\mathbf{G})$ grows. For systems where $\mathbf{G}(\mathbf{h})$ is continuous and recovery cost grows without bound as headroom vanishes—true for every system in this paper except TCP—the intermediate value theorem gives a critical headroom $\mathbf{h}^*$ where $\rho(\mathbf{G}(\mathbf{h}^*)) = 1$:

- $\mathbf{h}(t) > \mathbf{h}^*$: $\rho(\mathbf{G}) < 1 \to$ stable
- $\mathbf{h}(t) < \mathbf{h}^*$: $\rho(\mathbf{G}) > 1 \to$ cascade

The transition is one-directional. The system cannot self-rescue by becoming more overloaded.

B Computing h^*: UNIX Swap Thrashing

Swap crosses resource domains—it trades memory capacity (stock) for disk bandwidth (flow)—making it a clean test of the multi-resource framework.

Symbol	Meaning	Unit
S	Physical memory	pages
W	Total working set (all processes)	pages
$E = W - S$	Excess (pages that must swap)	pages
f	Distinct page access rate (all processes)	pages/sec
D'	Spare disk IOPS (total minus normal workload)	IOPS

[7] This follows from the Perron-Frobenius theorem for non-negative matrices: if $B \geq A$ entrywise, then $\rho(B) \geq \rho(A)$. The Collatz-Wielandt characterization $\rho(A) = \max_{x>0} \min_i (Ax)_i / x_i$ makes the monotonicity immediate.

Under uniform access, the probability of hitting a swapped-out page is E/W. Real workloads have locality: under the working set model [5], fault rate is near-zero while working sets fit in physical memory and spikes when they do not. The uniform assumption smooths this transition, yielding a lower bound on E^*—the true margin is larger, but the failure at the true boundary is sharper than the smooth model depicts. The table values below are conservative estimates of the minimum margin. Each fault costs 2 IOPS (swap-in read + swap-out write):

$$P = \frac{2fE}{W} \tag{6}$$

Setting $P = D'$ (disk saturation) and solving for the critical excess:

$$E^* = \frac{D'W}{2f} \tag{7}$$

For $W \approx S$, the swap margin ratio—excess as a fraction of physical memory:

$$\frac{E^*}{S} = \frac{D'}{2f} \tag{8}$$

Computed for a 64 GB server ($S = 16$M pages at 4 KB/page):

Workload	f (pages/sec)	HDD ($D' = 100$)	NVMe ($D' = 50{,}000$)
Light web (1K req/s)	10,000	320 MB (0.5%)	no h^*†
Database (1K qps)	100,000	32 MB (0.05%)	16 GB (25%)
Analytics scan	1,000,000	3.2 MB (0.005%)	1.6 GB (2.5%)

† $E^*/S > 1$: disk bandwidth exceeds maximum possible fault rate. The binding constraint shifts to latency (∼100 µs per NVMe fault), which the bandwidth model does not capture.

On a database server with HDD, the system thrashes at 32 MB of excess working set—0.05% over physical memory. On NVMe, 16 GB. The 500× IOPS improvement buys a proportional increase in margin, but the margin is finite and unmonitored.

Gain matrix at h^*. At disk saturation, the feedback loop:

$$\mathbf{G} = \begin{pmatrix} \varepsilon & \alpha \\ \beta & \varepsilon \end{pmatrix} \quad \text{(rows: disk, memory)} \tag{9}$$

- ε: self-coupling—each resource's recovery load feeding back on itself (I/O scheduler overhead, kernel swap bookkeeping). Small relative to the cross-resource terms; set equal for both resources as a simplification
- $\alpha = \partial(\text{swap IOPS})/\partial(-h_{\text{mem}}) \approx 2f/W$: each page of excess generates $2f/W$ IOPS
- $\beta = \partial(\text{mem demand})/\partial(-h_{\text{disk}})$: disk saturation blocks processes, which hold pages plus ∼8–16 KB of kernel I/O buffers per pending operation. Unlike α, β has no clean closed form—it depends on I/O scheduler behavior, process blocking semantics, and kernel buffer allocation. It is measurable empirically but not derivable from the parameters in this model

Eigenvalues: $\varepsilon \pm \sqrt{\alpha\beta}$. Cascade when $\alpha\beta \geq (1-\varepsilon)^2$—the cross-resource loop (memory pressure → swap → disk saturation → blocked processes → memory pressure) has compound gain ≥ 1.

C Computing h^*: TCP (The Degenerate Case)

TCP is the degenerate case: single resource (link bandwidth), scalar gain. The gain matrix is defined as a Jacobian (continuous), but TCP's backoff is a discrete iteration—each timeout event doubles the RTO. The "gain" here is the contraction ratio of that discrete map.

After k consecutive timeouts, RTO doubles: $\text{RTO}_k = \text{RTO}_{\text{base}} \times 2^k$. Retransmission rate:

$$r_k = \frac{1}{\text{RTO}_{\text{base}} \times 2^k} \tag{10}$$

When congestion increases (h decreases, another round of losses), k increments:

$$r_{k+1} = r_k/2 \tag{11}$$

$G_{\text{TCP}} = -0.5$. Each unit of congestion halves recovery demand.

The monotonicity result does not apply—G has a negative entry. $\rho(|G|) = 0.5 < 1$ at all loads. No h^* exists. TCP is unconditionally stable, not because recovery is bounded, but because it is a **contraction**: recovery reduces the condition that triggered it.

Scope. $G_{\text{TCP}} = -0.5$ is the contraction ratio of the backoff mechanism in isolation: a single flow's retransmission rate response to its own congestion signal. The system-level stability of TCP congestion control—including N-flow interaction, AIMD window dynamics, queue feedback, and RTT coupling—is a richer problem analyzed by Low et al. [15] using the same spectral framework. We use TCP here as a reference design exhibiting mechanism-level contraction, not as a claim about network-level stability.

	$G \geq 0$ (no backoff)	$G < 0$ (backoff)
$\rho(G)$ monotone in load	Non-decreasing	Can decrease
h^* exists	Always	May not
Self-rescue under overload	Impossible	Possible

D Computing h^*: Cassandra Hint Replay

Cassandra's cascade is disk-dominated: single resource (disk IOPS), scalar gain with a nonlinear trigger.

During an outage, hints accumulate at the write rate. On return, they replay at a throttled rate R, adding to normal disk load D_n. Disk utilization:

$$u = \frac{D_n + R}{D_t} \tag{12}$$

Read latency follows the M/M/1 queueing model:

$$L = \frac{b}{1-u} \tag{13}$$

where b is the base read latency under no contention. Timeouts occur when L exceeds the read timeout threshold τ:

$$u^* = 1 - \frac{b}{\tau} \tag{14}$$

At u^*, the feedback loop engages: timeouts trigger read repairs, each costing c_r IOPS, which increase utilization further. The feedback gain, derived via chain rule through the M/M/1 model:

$$g = \frac{c_r \cdot r_r \cdot \tau}{b \cdot D_t} \tag{15}$$

where r_r is the read rate. Below u^*, $g = 0$—no timeouts, no feedback. Above u^*, $g \gg 1$ for typical parameters. This is a phase transition, not a smooth crossing: the system jumps from stable to violently unstable at u^*.

The critical headroom—IOPS margin before cascade:

$$h^* = \frac{D_t \cdot b}{\tau} \tag{16}$$

The safe replay rate—maximum hint replay before crossing u^*:

$$R^* = D_t \left(1 - \frac{b}{\tau}\right) - D_n \tag{17}$$

Computed for a Cassandra node with $D_t = 500$ IOPS, $D_n = 200$ IOPS, $b = 5\,\text{ms}$, $\tau = 500\,\text{ms}$, $c_r = 3$ IOPS/repair, $r_r = 5{,}000$ reads/sec:

Quantity	Value	
u^*	0.99	99% utilization
h^*	5 IOPS	1% of disk capacity
R^*	295 ops/sec	safe replay rate
g (above u^*)	3,000	gain above threshold

At 5 IOPS of headroom—1% of a 500-IOPS disk—the read repair feedback gain jumps from zero to 3,000. The default Cassandra throttle (128 ops/sec) is safe; raising it to 300 is cascade.

Structural contrast with swap. Both h^* values are closed-form functions of measurable quantities. The instability structures differ: swap has a smooth transition (ρ crosses 1 as load increases; the linearized h^* is a conservative lower bound). Cassandra has a phase transition (gain jumps from 0 to $\gg 1$ at u^*). The M/M/1 nonlinearity creates a cliff, not a slope. The linearization critique (Hartman-Grobman radius unknown) applies to swap's smooth boundary but not to Cassandra's discontinuous one—the phase transition *is* the nonlinearity, computed exactly.

E The OOM Killer: Why h^* Cannot Be Computed

The OOM killer operates on a single resource (memory), so the gain is scalar. When the kernel kills a process, it frees m_f bytes. If a supervisor restarts the process, it allocates m_r bytes during initialization. The gain ratio:

$$G_{\text{oom}} = \frac{m_r}{m_f} \tag{18}$$

When $G_{\text{oom}} < 1$, the kill-restart cycle converges: each iteration frees more memory than the restart consumes. When $G_{\text{oom}} \geq 1$, the restart allocates at least as much as the kill freed, recreating the pressure that triggers the next kill. The cascade condition is $G_{\text{oom}} = 1$.

The kernel cannot evaluate G_{oom}. The denominator m_f is observable—the kernel knows how much memory a killed process held. The numerator m_r depends on process behavior the kernel

does not model: JVM heap reservation, application startup sequences, supervisor restart policies, memory allocation profiles during initialization. The `oom_badness` heuristic selects *which* process to kill based on current memory footprint—estimating m_f—but has no model of m_r.

This is a level 0 violation: the kernel lacks the resource model for the restart consequence of its own action. The gain ratio is not merely unmeasured—it is unmeasurable by the acting entity without a model of process startup behavior.

The framework's prescription follows directly: when you cannot compute h^*, you cannot prove recovery is safe. The correct action is to emit diagnostic state—which process was killed, system memory state, recent allocation rates—and shut down. Contrast with swap (section B), where the kernel owns both sides of the gain matrix (page fault rate and disk I/O), and with Cassandra (section D), where the node owns the disk utilization model. The OOM killer is the only case study in this paper where the gain ratio is formally unknowable to the entity that acts on it.

F Note on Nonlinearity

The gain matrix $\mathbf{G}$ is the Jacobian of the recovery dynamics at the current operating point—a local linearization. The nonlinear system's behavior near this point is governed by two results.

The linearization is locally valid when no eigenvalue of $\mathbf{G}$ lies on the unit circle: $\rho(\mathbf{G}) < 1$ implies local asymptotic stability, $\rho(\mathbf{G}) > 1$ implies local instability.[8]

For $\mathbf{G} \geq 0$, the monotonicity of $\rho(\mathbf{G})$ in load (established in section A) means the boundary $\mathbf{h}^*$ is a clean phase transition. The nonlinear system may reach instability sooner than the linearization predicts—a large perturbation can cross $\mathbf{h}^*$ even from a locally stable operating point—but once past $\mathbf{h}^*$ it cannot self-rescue. The linear analysis never gives false instability warnings: if $\rho(\mathbf{G}) \geq 1$, the system *will* cascade. But it can give false assurance of robustness when the basin of attraction is smaller than the perturbation. Resistance to large perturbations requires headroom margin beyond $\mathbf{h}^*$.[9]

Acknowledgments

Thanks to Anton Kast for feedback on the mathematical presentation.

References

[1] Peter Alvaro, Rebecca Isaacs, Rupak Majumdar, Kiran-Kumar Muniswamy-Reddy, Mahmoud Salamati, and Sadegh Soudjani. Formal analysis of metastable failures in software systems. *arXiv preprint arXiv:2510.03551*, 2025.

[2] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. Metastable failures in distributed systems. In *HotOS '21: Workshop on Hot Topics in Operating Systems*, 2021.

[3] Tushar Deepak Chandra and Sam Toueg. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2):225–267, 1996.

[8] This is the discrete Hartman-Grobman theorem [9, 8, 10]: the nonlinear system's local phase portrait is topologically equivalent to its linearization at a hyperbolic fixed point.

[9] For exact convergence rates and dynamics near $\rho(\mathbf{G}) = 1$, Lyapunov stability analysis and the theory of cooperative dynamical systems [12, 18] provide stronger quantitative bounds.

[4] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. In *SOSP '07: Proceedings of the 21st ACM Symposium on Operating Systems Principles*, 2007.

[5] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[6] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.

[7] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, San Mateo, CA, 1993.

[8] D. M. Grobman. Homeomorphisms of systems of differential equations. *Doklady Akademii Nauk SSSR*, 128:880–881, 1959.

[9] Philip Hartman. A lemma in the theory of structural stability of differential equations. *Proceedings of the American Mathematical Society*, 11:610–620, 1960.

[10] Philip Hartman. On local homeomorphisms of euclidean spaces. *Boletín de la Sociedad Matemática Mexicana*, 5:220–241, 1960.

[11] Joseph L. Hellerstein, Yixin Diao, Sujay Parekh, and Dawn M. Tilbury. *Feedback Control of Computing Systems*. Wiley, Hoboken, NJ, 2004.

[12] Morris W. Hirsch. Systems of differential equations that are competitive or cooperative II: Convergence almost everywhere. *SIAM Journal on Mathematical Analysis*, 16(3):423–439, 1985.

[13] Lexiang Huang, Matthew Magnaye, Anand Shenoy, Jialun Hu, Aleksey Charapko, Abutalib Aghayev, and Timothy Zhu. Metastable failures in the wild. In *OSDI '22: 16th USENIX Symposium on Operating Systems Design and Implementation*. USENIX, 2022.

[14] Rebecca Isaacs, Peter Alvaro, Rupak Majumdar, Kiran Reddy, Mahmoud Salamati, and Sadegh Soudjani. Analyzing metastable failures. In *HotOS '25: Workshop on Hot Topics in Operating Systems*, pages 172–178, 2025.

[15] Steven H. Low, Fernando Paganini, and John C. Doyle. Internet congestion control. *IEEE Control Systems Magazine*, 22(1):28–43, 2002.

[16] Stefanny Ramirez, Maaike Odijk, and Dario Bauso. Cascading failures: Dynamics, stability and control. *arXiv preprint arXiv:2305.00838*, 2023.

[17] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A comprehensive study of convergent and commutative replicated data types. Technical Report RR-7506, INRIA, 2011.

[18] Hal L. Smith. *Monotone Dynamical Systems: An Introduction to the Theory of Competitive and Cooperative Systems*. Number 41 in Mathematical Surveys and Monographs. American Mathematical Society, Providence, RI, 1995.

[19] David D. Woods. Four concepts for resilience and the implications for the future of resilience engineering. *Reliability Engineering and System Safety*, 141:5–9, 2015.

[20] David D. Woods. The theory of graceful extensibility: Basic rules that govern adaptive systems. *Environment Systems and Decisions*, 38:433–457, 2018.

Thanks for reading. If you work on distributed systems and have seen the recovery invariant violated in ways not covered here, or if you spot errors in the formalization, I'd like to hear about it.